SUBHAPREET PATRO
2211CS010547

DL Assignment-4

Q1) What is a Convolutional Neural Network? How does it work?

A)

Convolutional Neural Network (CNN)

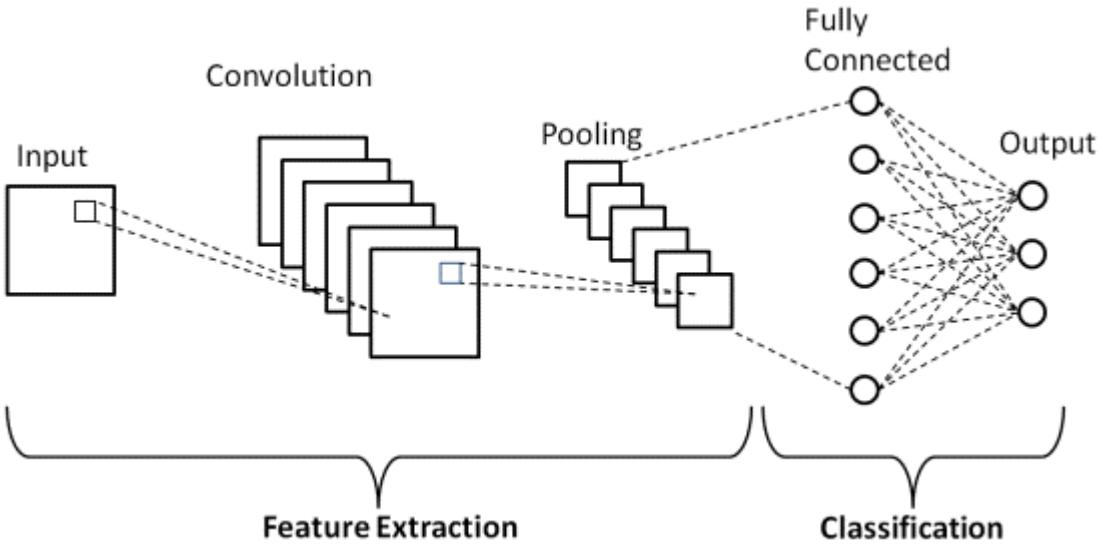
A **Convolutional Neural Network (CNN)** is a type of **Artificial Neural Network (ANN)** specifically designed to process **structured grid-like data**, such as images (2D grids of pixels) or even time series data (1D sequences).

Unlike traditional feed-forward neural networks, which treat input data as a flat vector, CNNs **preserve the spatial structure of data**. This makes them especially effective for **image recognition, object detection, natural language processing, and video analysis**.

CNNs are inspired by the **visual cortex** in the human brain, which processes visual information in a hierarchical way — detecting edges, shapes, textures, and then complex objects.

How does a CNN work?

CNNs are built from a series of **specialized layers** that automatically and adaptively learn spatial features from data.



Here's the step-by-step working:

1. Input Layer

- The input to a CNN is usually an **image** represented as a **3D tensor**:

Width×Height×Channels

For example, a **32×32 RGB image** has size 32×32×3.

- The channels correspond to **color layers** (Red, Green, Blue).

2. Convolutional Layer

- The **core building block** of a CNN.
- Uses **filters (kernels)**: small learnable matrices (e.g., 3×3 or 5×5).
- Each filter slides over the input image performing a **convolution operation**:

Feature Map=Input∗Filter\text{Feature Map} = \text{Input} * \text{Filter}

- The filter detects **patterns** (edges, corners, textures).
- Multiple filters produce multiple **feature maps**, each focusing on different features.

Example:

- One filter may detect vertical edges.
- Another may detect horizontal edges.
- Another may detect color intensity.

This is how CNNs **automatically learn** useful features.

3. Activation Function (ReLU)

- After convolution, the output passes through an **activation function**.
- The most common is **ReLU (Rectified Linear Unit):**

f(x)=max(0,x)

- Purpose:
    - Introduces **non-linearity**.
    - Ensures the network can learn complex patterns.
    - Prevents negative values from interfering with feature representation.

4. Pooling Layer (Downsampling)

- Reduces the **spatial size** of the feature maps while keeping the most important information.
- Types of pooling:
    - **Max Pooling**: takes the maximum value in each region (most common).
    - **Average Pooling**: takes the average.
- Benefits:
    - **Reduces computation**.

- Makes features **invariant to translation** (object location doesn't matter as much).
- Prevents overfitting.

Example:
A 2×2 max pooling reduces a 4×4 feature map into a 2×2 by selecting the max value in each block.

5. Stacking Layers

- CNNs usually consist of **multiple convolution + pooling layers stacked together**.
- Early layers → detect **low-level features** (edges, textures).
- Deeper layers → detect **high-level features** (shapes, objects, faces).

6. Flattening

- After several convolution and pooling operations, the final feature maps are **flattened into a 1D vector**.
- Example: From a 7×7×64 feature map → flatten to a vector of size 3136.

7. Fully Connected Layers (Dense Layers)

- Acts like a traditional ANN.
- Every neuron is connected to every feature.
- Combines learned features into a **final decision**.
- Usually ends with a **softmax function** (for classification problems), which outputs probabilities for each class.

8. Output Layer

- Produces the final prediction.
- For image classification:
  - Each neuron corresponds to a class (e.g., cat, dog, car).
  - Softmax ensures probabilities sum to 1.

Training a CNN

CNNs are trained using **backpropagation** with **gradient descent**.

- The network makes a prediction.
- Loss function (e.g., cross-entropy) compares prediction vs. actual label.

- Gradients are computed w.r.t filters and weights.
- Filters adjust themselves to improve feature detection.
- Over time, the CNN learns **optimal filters** for the task.

Example Workflow (Image Classification)

1. Input: A picture of a dog.
2. Convolution Layer: Detects edges, fur patterns, shapes.
3. Pooling Layer: Downsamples, keeps important details.
4. More Convolutions: Recognizes ears, eyes, tail.
5. Flatten → Fully Connected Layer: Combines all features.
6. Output: Probability that the image is a **dog** (e.g., 95%).

Applications of CNNs

- Image & Video Recognition (face recognition, object detection).
- Medical Imaging (tumor detection, X-ray classification).
- Self-driving Cars (road/lane detection).
- Natural Language Processing (text classification, sentiment analysis).
- Satellite Image Processing.

Q2) What is the concept of a Recurrent Neural Network, and in what way does it operate?

A)

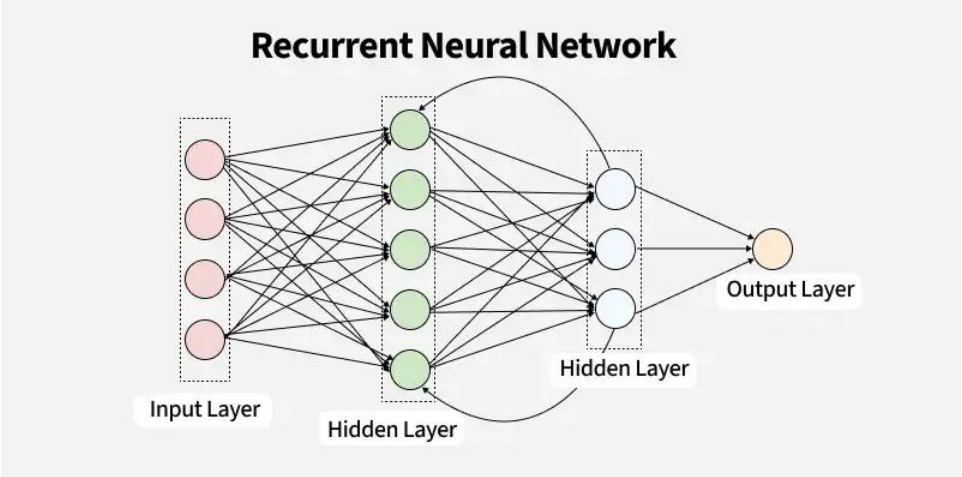
What is a Recurrent Neural Network (RNN)?

A **Recurrent Neural Network (RNN)** is a class of neural network designed specifically for **sequential data** — anything where the order of elements matters (text, speech, time series, video frames).
Unlike feed-forward networks, an RNN has a **recurrent connection** that lets it keep a hidden state (memory) that summarizes information seen so far. That hidden state is updated step-by-step as new inputs arrive, so the model can capture temporal dependencies and patterns across time.

SUBHAPREET PATRO
2211CS010547

Intuition — why recurrence helps

Imagine reading a sentence word by word. To understand the meaning of the current word you use what you read before. An RNN mimics that: at time step t it receives an input x_t and the previous hidden state h_{t-1}, and produces a new hidden state h_t which encodes both new input and context from the past. This makes RNNs useful for tasks like language modeling, speech recognition, and forecasting.



The basic (vanilla) RNN: forward equations

At each time step t:

ht=ϕ(Whhht−1+Wxhxt+bh)h_t = \phi(W_{hh} h_{t-1} + W_{xh} x_t + b_h)
yt=g(Whyht+by)y_t = g(W_{hy} h_t + b_y)

- x_t = input at time t (vector)
- h_t = hidden state at time t (vector)
- y_t = output at time t (vector)
- W_{xh}, W_{hh}, W_{hy} = weight matrices; b_* biases
- ϕ = nonlinearity (often tanh or ReLU)
- g = output activation (softmax for classification)

You can **unroll** the RNN across time steps to visualize the computation as a feed-forward graph where weights are shared at every time step.

How training works — Backpropagation Through Time (BPTT)

- Define a loss (e.g., cross-entropy) across the sequence: L=∑tℓ(yt,targett)L = \sum_t \ell(y_t, target_t).

- Backpropagation is applied on the unrolled graph: gradients flow backward through time to update shared weights.
- This is called **Backpropagation Through Time (BPTT)**.
- For long sequences, we often use **truncated BPTT** (backprop for a limited number of steps) for efficiency and numerical stability.

Main practical problems

1. **Vanishing gradients** — gradients shrink exponentially over many time steps → network struggles to learn long-range dependencies.
2. **Exploding gradients** — gradients grow exponentially → numerical instability.

Common mitigations:

- Use gated RNN cells (LSTM/GRU) which help preserve gradients.
- Gradient clipping to cap large gradients.
- Careful initialization, layer normalization, better optimizers (Adam, RMSProp).

Gated RNNs: LSTM and GRU (how they operate)

Gated cells were invented to overcome vanishing gradients by providing controlled paths for gradient flow and selective memory.

LSTM (Long Short-Term Memory)

Tracks two vectors: **cell state** c_t (long-term memory) and **hidden state** h_t (output). It uses three gates (forget, input, output) plus a candidate update:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad \text{(forget gate)}$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad \text{(input gate)}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad \text{(candidate)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad \text{(output gate)}$$

$$h_t = o_t \odot \tanh(c_t)$$

Gates (sigmoid outputs in [0,1]) decide what to keep, write, and reveal — which helps preserve information over long intervals.

GRU (Gated Recurrent Unit)

A simpler cell that merges gates and has fewer parameters:

$$z_t = \sigma(W_z x_t + U_z h_{t-1}) \quad \text{(update gate)}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1}) \quad \text{(reset gate)}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRUs are computationally cheaper and often perform comparably to LSTMs.

RNN architectures and sequence types

- **Many-to-one**: sequence → single output (e.g., sentiment classification)
- **One-to-many**: single input → sequence output (e.g., image captioning)
- **Many-to-many (aligned)**: input and output sequences same length (e.g., POS tagging)
- **Many-to-many (seq2seq)**: input and output sequences different lengths (e.g., machine translation) — often implemented with **encoder–decoder** RNNs; attention greatly improves performance by letting the decoder focus on relevant encoder states.

Bidirectional RNNs process the sequence in forward and backward directions and concatenate states so the model sees left and right context — useful when the whole sequence is available (e.g., labeling tasks).

SUBHAPREET PATRO
2211CS010547

Practical considerations & tips

- **Variable-length sequences**: use padding + masking or packed sequences (framework-specific) to batch efficiently.
- **Teacher forcing** (during training for sequence generation): feed the true previous token to decoder; at inference use predicted token — affects training dynamics.
- **Batching**: organize similar-length sequences to reduce padding overhead.
- **Regularization**: dropout (careful where to apply), layer normalization, weight decay.
- **Optimization**: Adam/RMSProp often work well; gradient clipping helps prevent explosion.
- **When not to use RNNs**: for very long-range dependencies or large datasets, **Transformers** (self-attention) often outperform RNNs and are more parallelizable.

Use cases

- Language modeling & translation
- Speech recognition and synthesis
- Time series forecasting (finance, sensors)
- Handwriting recognition, music generation
- Video / frame sequence analysis (short-term temporal patterns)